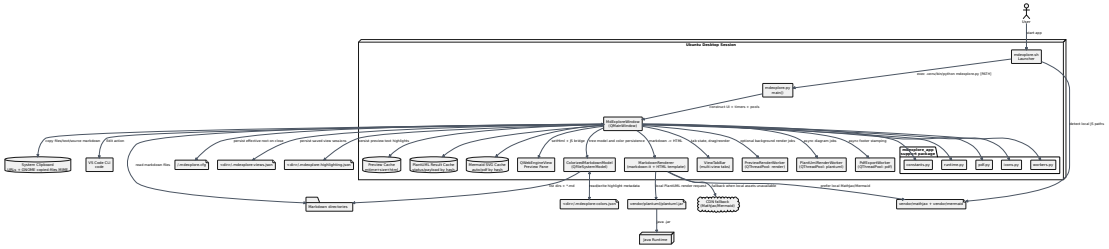
mdexplore UML

This document provides a comprehensive PlantUML view of the current `mdexplore` implementation. All diagrams are embedded so they can be rendered directly by mdexplore (or any Markdown viewer with PlantUML support).

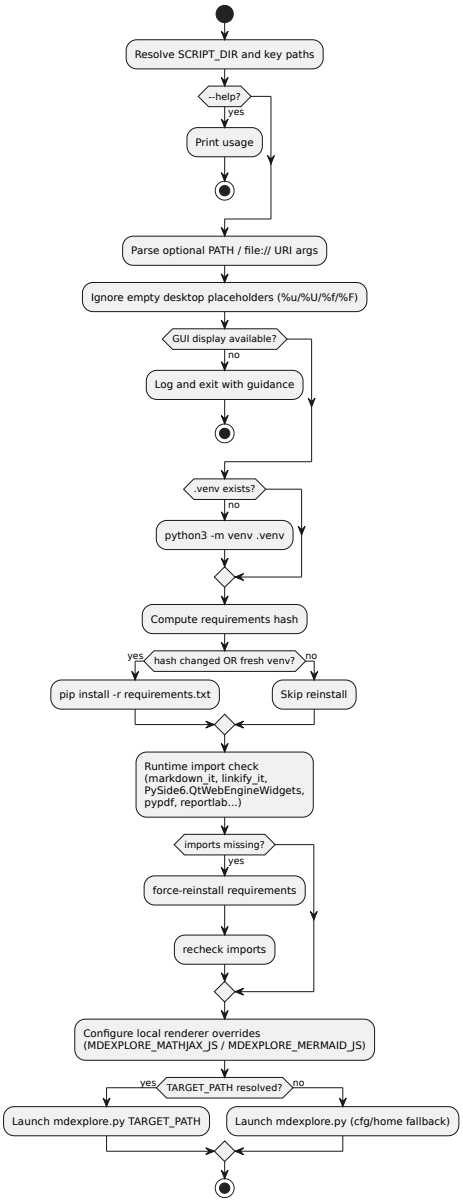
Detailed render/caching forks (GUI vs PDF, JS Mermaid vs Rust Mermaid, cache mode ownership, and restore behavior) are intentionally documented in `RENDER-PATHS.md`. This UML file keeps those areas abstracted at system/class boundaries to avoid duplicating low-level render logic across two docs.

Low-risk modularization note: support code now lives in `mdexplore_app/` (`constants.py`, `runtime.py`, `pdf.py`, `icons.py`, `workers.py`), while the main orchestration, renderer/template logic, and UI state machine remain in `mdexplore.py`. The diagrams below show that split at subsystem boundaries rather than expanding every extracted helper inline.

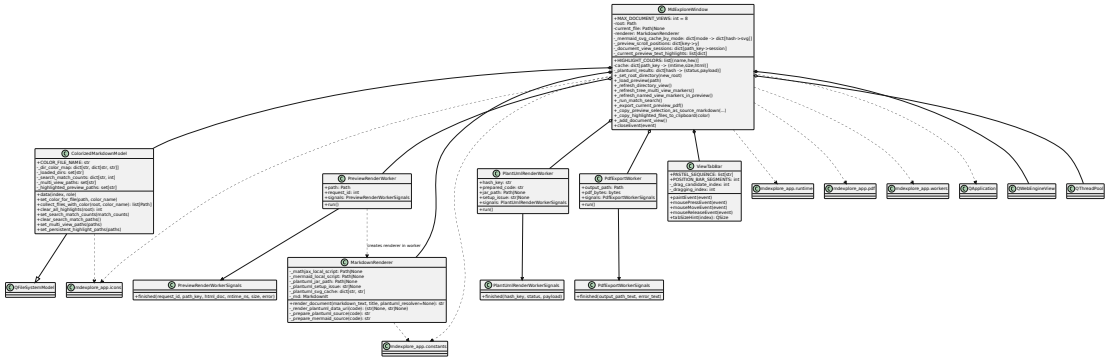
1. System Architecture


User
open app
Ubuntu Desktop Session
mdeditor.sh
exec: /usr/local/python mdeditor.py [file1.md]
mdeditor.py
main()
Construct UI + Gtkmain +: pseudo
MdEditorWindow
(GtkApplicationWindow)
Gtk.Clipboard
(default display)
text: Text/plain/uri-list/utf8
VS Code CSS
code
read markdown files
/usr/share/plantuml
/usr/share/mdeditor/views.json
/usr/share/mdeditor/highlighting.conf
dirtyset: set on close
dirtyset: reset view selection
debounce 300 ms
preset preview text: highlight
Preview Center
(WebKitWebView)
Theme(0): Result Center
(WebKitWebView)
Theme(1): Edit Center
(WebKitWebView)
GtkMainLoop(idle)
Preview Panel
UrlSchemeHandlerBlocker
(GtkPlugSocketWindow)
MarkdownRenderer
(mistune + 3 HTML templates)
ViewTabBar
(Notebook view tabs)
PreviewRendererWorker
(GThreadPool: render)
PlantUmlRendererWorker
(GThreadPool: plantuml)
MdEditorWorker
(GThreadPool: md)
Mdeditor.app
(Gtk.Application)
/usr/share/.../
mdeditor.svg
/usr/share/.../
mdeditor.desktop
/usr/share/.../
mdeditor.1.gz
/usr/share/.../
mdeditor.1.gz
/usr/share/.../
mdeditor.1.gz

2. Launcher Runtime Activity (`mdexplore.sh`)

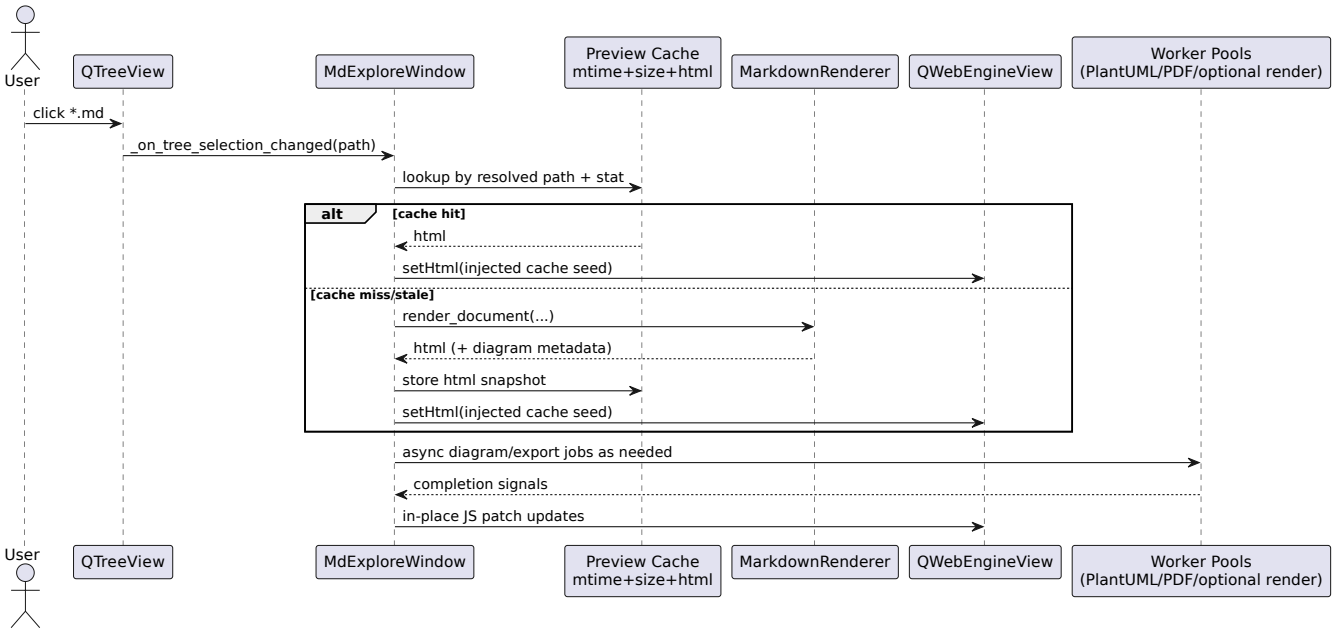


3. Core Class Diagram



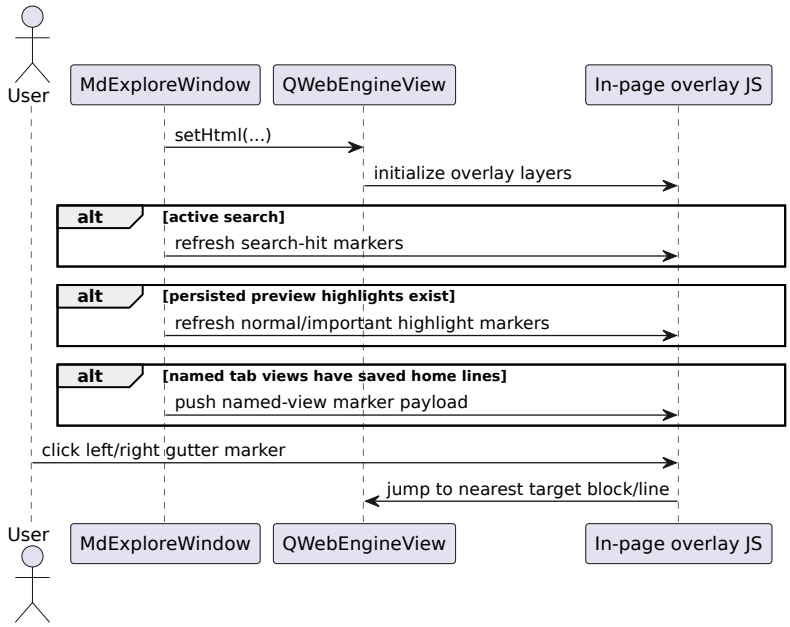
4. Preview Pipeline (Abstracted)

The detailed render/cache branches are covered in `RENDER-PATHS.md`. This section intentionally keeps an architectural boundary view only.

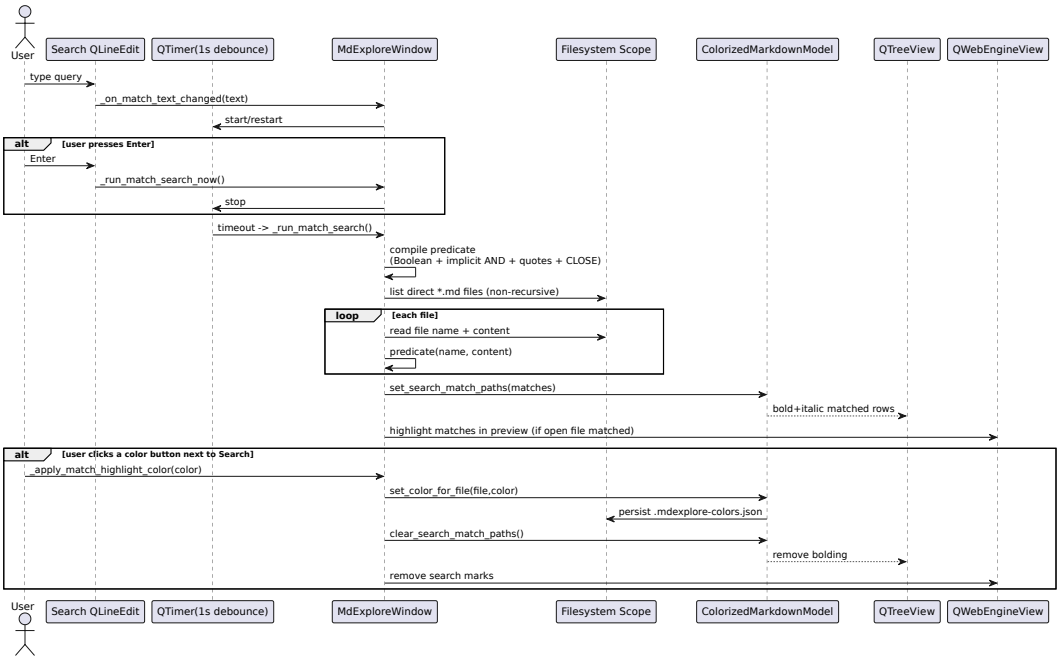


4.1 Preview Navigation Overlays

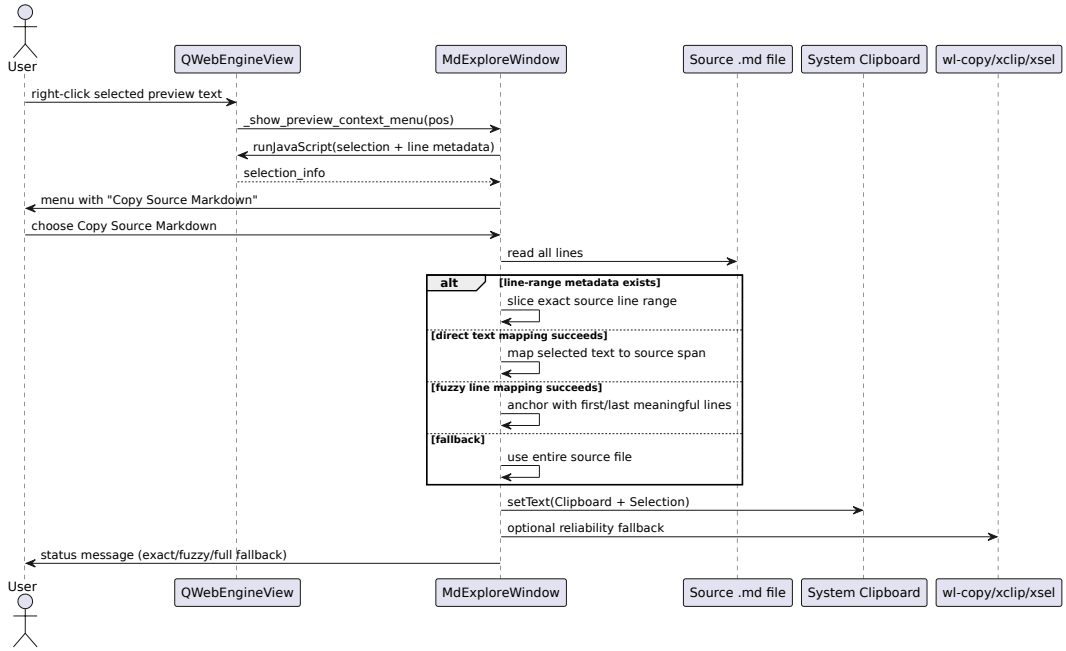
These are intentionally shown here only as boundary-level UI overlays. The exact DOM/CSS/JS mechanics remain in `RENDER-PATHS.md`.



5. Search + Highlight + Apply Color Flow

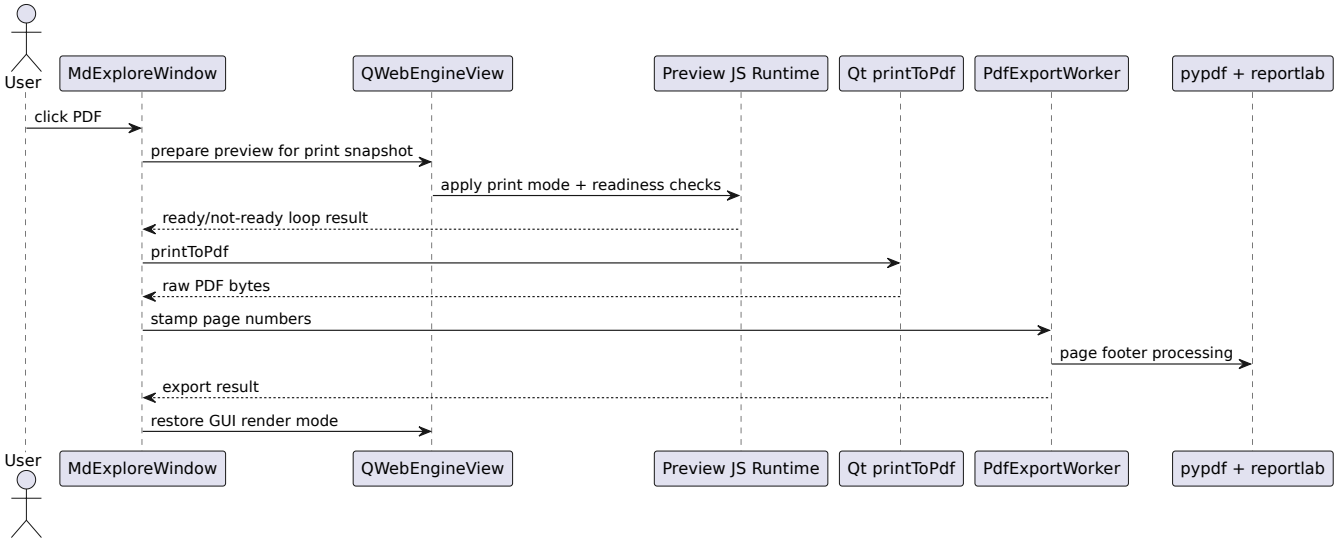


6. Preview Context Menu: Copy Source Markdown

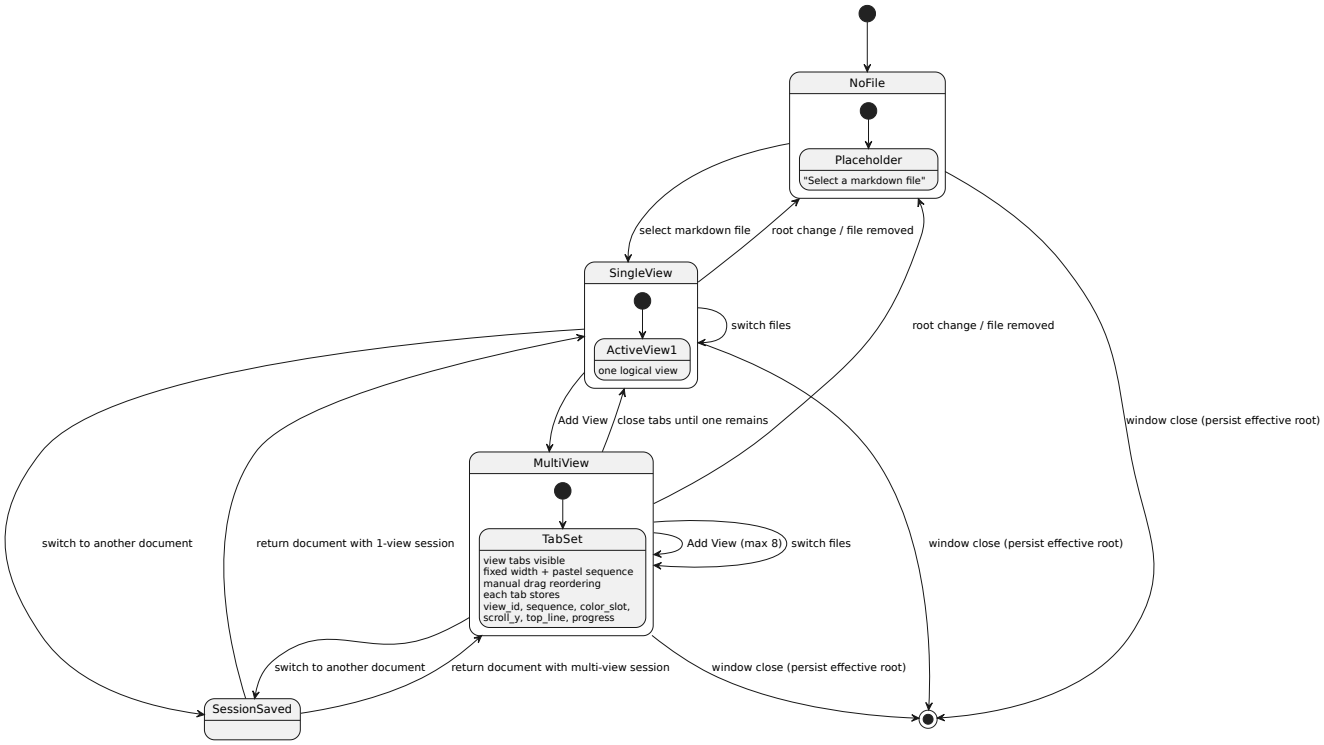


7. PDF Export Pipeline (Abstracted)

Detailed PDF mode branching (JS Mermaid grayscale path vs Rust Mermaid default-themed PDF cache path) is documented in `RENDER-PATHS.md`.



8. Document View/Tab Lifecycle



Notes

- Diagrams are based on current code in `mdexplore.py` and `mdexplore.sh`.
- Render/caching branch internals are intentionally abstracted here and documented in `RENDER-PATHS.md` to keep a single authoritative deep map.
- Preview-only zoom (`Ctrl++` , `Ctrl+-` , `Ctrl+0`) is also intentionally kept out of the UML internals here because it is a `QWebEngineView` scale adjustment, not a separate renderer/cache branch.
- Preview gutter overlays (search-hit markers, persistent-highlight markers, named-view home markers) are similarly abstracted here and described in more detail in `RENDER-PATHS.md`, because they are post-render navigation aids rather than renderer/cache forks.
- Worker/threadpool usage is intentionally separated by concern:
  - render pool (preview HTML jobs),
  - PlantUML pool (diagram jobs),
  - PDF pool (post-processing/stamping).
- PlantUML rendering is non-blocking in UI flow: placeholders are rendered first, then patched in place as jobs finish.
- TODO (known issue): diagram zoom/pan restore (Mermaid and PlantUML) is not yet consistently reliable when leaving a document and returning in the same app run.